

Metody Optymalizacji

Projekt: CVRPTW

Jakub Wasilewski 263852

19 czerwca 2026

Spis treści

1	Sformułowanie zadania	3
2	Sposób rozwiązywania zadania	3
2.1	Kodowanie rozwiązania	3
2.2	Ocena rozwiązania	3
3	Implementacja	4
3.1	Algorytm ewolucyjny	4
3.2	Operatory niestandardowe	4
4	Pliki wejściowe	5
5	Procedura badawcza	5
5.1	Strojenie parametrów	5
5.2	Porównanie końcowe	6
5.3	Środowisko obliczeniowe	6
6	Strojenie parametrów – potok ops_on	7
6.1	Faza 1: rozmiar populacji	7
6.2	Faza 2: operator krzyżowania	7
6.3	Faza 3: prawdopodobieństwo krzyżowania (Xp)	8
6.4	Faza 4: prawdopodobieństwo mutacji (Mp)	9
6.5	Faza 5: rozmiar turnieju (turSize)	10
6.6	Faza 6: prawdopodobieństwo Repair (REPAIRp)	11
6.7	Faza 7: prawdopodobieństwo OptimizeTracks (OPTp)	12
6.8	Faza 8: prawdopodobieństwo RedistributeLocations (REDISTp)	13
6.9	Najlepsze parametry – potok ops_on	14
7	Strojenie parametrów – potok ops_off	14
7.1	Faza 1: rozmiar populacji	15
7.2	Faza 2: operator krzyżowania	15
7.3	Faza 3: prawdopodobieństwo krzyżowania	16
7.4	Faza 4: prawdopodobieństwo mutacji	17
7.5	Faza 5: rozmiar turnieju	17
7.6	Najlepsze parametry – potok ops_off	18
8	Porównanie końcowe	18
8.1	Przebiegi algorytmu	20

9	Analiza wyników i wnioski	21
9.1	Wpływ operatorów niestandardowych	21
9.2	Kluczowe obserwacje	22
9.3	Porównanie z najlepszymi znanymi rozwiązaniami (BKS)	22
9.4	Ograniczenia badania	23

1 Sformułowanie zadania

Celem zadania była implementacja i eksperymentalne zbadanie algorytmu ewolucyjnego (EA) dla *Capacitated Vehicle Routing Problem with Time Windows* (CVRPTW) – pojemnościowego problemu wyznaczania tras pojazdów z oknami czasowymi.

W CVRPTW rozpatrywano n klientów i flotę pojazdów o jednakowej pojemności Q , startujących i kończących trasę w jednym magazynie (depot). Każdy klient $i \in \{1, \dots, n\}$ charakteryzowany był przez:

- lokalizację (x_i, y_i) ,
- zapotrzebowanie $d_i > 0$,
- okno czasowe $[e_i, l_i]$ – najwcześniejszy i najpóźniejszy dopuszczalny czas obsługi,
- czas obsługi s_i .

Celem było znalezienie zestawu tras minimalizujących **łączną długość tras** przy zachowaniu ograniczeń pojemnościowych i okien czasowych. Jeśli pojazd przybywa do klienta przed e_i , czeka bez kary. Przekroczenie l_i karane jest proporcjonalnie do opóźnienia (miękkie ograniczenie).

2 Sposób rozwiązywania zadania

2.1 Kodowanie rozwiązania

Rozwiązanie kodowane jest jako permutacja liczb całkowitych o długości $L = \text{SIZE} + \text{MAX_VEHICLES} - 1$.

- Wartości $0 \dots \text{SIZE} - 1$ oznaczają klientów.
- Wartości $\geq \text{SIZE}$ są separatorami tras („powrót do bazy”).

Dla $n = 5$ klientów i 3 pojazdów ($L = 7$) przykładowy genom:

$$x = [3, 1, \mathbf{5}, 0, 4, \mathbf{6}, 2]$$

odpowiada trasom: $\{3, 1\} \rightarrow \text{baza} \rightarrow \{0, 4\} \rightarrow \text{baza} \rightarrow \{2\}$. Separatory ≥ 5 są ignorowane pod względem tożsamości – liczy się tylko ich pozycja w genomie; wartości klientów muszą tworzyć permutację $\{0, \dots, n - 1\}$.

Dla instancji Solomona (100 klientów, 25 pojazdów): $L = 100 + 25 - 1 = 124$.

2.2 Ocena rozwiązania

Funkcja `EstimateSolution` symuluje przejazd każdego pojazdu od lewej do prawej w genomie:

1. Pojazd startuje z bazy ($t = 0$, ładunek = 0).
2. Po napotkaniu separatora: reset czasu i ładunku (powrót do bazy).
3. Po napotkaniu klienta i : sprawdzany jest ładunek (jeśli d_i przekracza pozostałą pojemność, automatycznie wraca do bazy), aktualizowany jest czas przyjazdu, doliczana kara za spóźnienie, dodawany czas obsługi s_i .
4. Koszt całkowity = suma odległości euklidesowych + suma kar.

Kary:

- Spóźnienie ($t > l_i$): $\text{LATE_ARRIVAL_PENALTY} = 0,01 \times (t - l_i)$.
- Wczesne przybycie ($t < e_i$): brak kary, pojazd czeka.

Miękkie ograniczenia umożliwiają algorytmowi eksplorację przestrzeni rozwiązań niekoniecznie dopuszczalnych, co ułatwia przejście przez obszary zakazane.

3 Implementacja

3.1 Algorytm ewolucyjny

Zaimplementowano klasyczny algorytm ewolucyjny z następującymi składowymi:

Inicjalizacja Populacja P losowych permutacji generowana metodą Fisher-Yates ($\mathcal{O}(n)$).

Selekcja Turniejowa: losuje się $k = \text{turSize}$ osobników i wybiera tego z najmniejszą wartością funkcji celu. Mała wartość k zwiększa presję selekcyjną łagodnie; duża szybko eliminuje słabych osobników.

Krzyżowanie Stosowane z prawdopodobieństwem X_p . Dostępne operatory:

- **OX** (*Ordered Crossover*) – kopiuje ciągły segment z P1, uzupełnia brakujące geny w kolejności z P2. Zachowuje względną kolejność.
- **PMX** (*Partially Mapped Crossover*) – definiuje odwzorowanie między segmentami rodziców i przenosi je na potomka.
- **CX** (*Cycle Crossover*) – identyfikuje cykle pozycjonalne i kopiuje geny cyklicznie z P1 i P2.

Mutacja Stosowana z prawdopodobieństwem M_p :

- **SWAP** – zamiana dwóch losowych genów.
- **INVERSE** – odwrócenie losowego segmentu genomu.

Elitaryzm elitesNum najlepszych osobników przechodzi do nowej populacji bez zmian – gwarantuje monotoniczność najlepszego rozwiązania.

Nowa populacja Generowana przez selekcję + krzyżowanie + mutacja dla pozostałych $P - \text{elitesNum}$ miejsc.

3.2 Operatory niestandardowe

Po standardowej mutacji z określonymi prawdopodobieństwami stosowane są trzy dodatkowe operatory, działające na poziomie tras:

Repair (naprawa okien czasowych): Operator przechodzi genom i identyfikuje klientów, dla których czas przyjazdu przekracza l_i (spóźnienie). Klientów naruszających okno czasowe wycina z bieżącej pozycji i próbuje wstawić ich w inne miejsce genomu, gdzie ograniczenie jest spełnione. Jeśli żadna pozycja nie eliminuje naruszenia, klient wstawiany jest w miejsce minimalizującym karę. Operator poprawia dopuszczalność rozwiązania bez ingerencji w ogólną strukturę podziału na trasy.

Parametr: REPAIR_p – prawdopodobieństwo zastosowania operatora.

OptimizeTracks (lokalna optymalizacja kolejności w trasie): Dla każdej trasy z osobna wykonuje zachłanne wstawianie (*greedy insertion*): buduje trasę od nowa, w każdym kroku wybierając klienta minimalizującego bieżący koszt cząstkowy. Operator nie przenosi klientów między trasami – optymalizuje wyłącznie kolejność obsługi w obrębie jednej trasy.

Parametr: OPTp – prawdopodobieństwo zastosowania operatora.

RedistributeLocations (redystrybucja klientów między trasami): Wybiera losową trasę-dawcę i losowego klienta; próbuje przenieść go na każdą możliwą pozycję w każdej innej trasie (biorcy). Akceptuje ruch, jeśli obniża łączny koszt. Operator optymalizuje podział klientów między trasy, czego nie potrafią krzyżowanie ani standardowa mutacja (operują na całym genomie).

Parametry: REDISTp – prawdopodobieństwo zastosowania; REDIST_TRIES – liczba losowanych par dawca–klient.

Kolejność stosowania: Operatory stosowane są sekwencyjnie po mutacji standardowej: RedistributeLocations → Repair → OptimizeTracks. Kolejność ma znaczenie: redystrybucja zmienia strukturę tras, naprawa eliminuje naruszenia w nowych trasach, optymalizacja dopracowuje kolejność po naprawie.

4 Pliki wejściowe

Eksperymenty przeprowadzono na instancjach z zestawu benchmarkowego Solomona (100 klientów). Zestaw Solomona dzieli się na 6 rodzin w zależności od rozmieszczenia klientów i charakteru okien czasowych:

Rodzina	Rozmieszczenie klientów	Okna czasowe
C1	klastrowe	wąskie
C2	klastrowe	szerokie
R1	losowe	wąskie
R2	losowe	szerokie
RC1	mieszane	wąskie
RC2	mieszane	szerokie

Do strojenia parametrów użyto 3 instancji (po jednej z C1, R1, RC1): **C101, R101, RC101**.

Do porównania końcowego użyto 12 instancji – po dwie z każdej rodziny: C101, C102, C201, C202, R101, R102, R201, R202, RC101, RC102, RC201, RC202.

5 Procedura badawcza

5.1 Strojenie parametrów

Strojenie przeprowadzono metodą **jednowymiarowej analizy wrażliwości** (*one-factor-at-a-time*): w każdej fazie zmieniany jest jeden parametr, pozostałe utrzymywane na wartości wybranej w poprzedniej fazie (tzw. *sequential tuning*).

Kryterium zatrzymania podczas strojenia: stały budżet $B = 300\,000$ wywołań funkcji oceny. Każda konfiguracja testowana $N = 5$ razy niezależnie.

Miary jakości stosowane w tabelach strojenia:

- Wartość dla instancji (np. kolumna **C101**) – średnia arytmetyczna wartości funkcji oceny najlepszego osobnika z ostatniej generacji, uśredniona po $N = 5$ niezależnych uruchomieniach.

- **OVERALL** – średnia arytmetyczna wartości z trzech instancji strojenia (C101, R101, RC101); używana jako kryterium wyboru zwycięzcy fazy.

Odchylenie standardowe nie jest wyznaczane podczas strojenia ze względu na małą liczbę powtórzeń ($N = 5$) – służy ono wyłącznie do selekcji wariantów, nie do wnioskowania statystycznego.

Przeprowadzono **dwa niezależne potoki strojenia**:

- **ops_on**: EA z włączonymi operatorami niestandardowymi – 8 faz.
- **ops_off**: EA bazowy bez operatorów niestandardowych – 5 faz.

Każdy potok startuje z ustalonej **konfiguracji bazowej**, od której odchylany jest jeden parametr na raz. Konfiguracje bazowe obu potoków przedstawia tabela 1.

Tabela 1: Konfiguracje bazowe obu potoków strojenia

Parametr	ops_on (bazowa)	ops_off (bazowa)
popSize	50	50
Krzyżowanie	OX	OX
Xp	75%	75%
Mp	25%	25%
turSize	2	2
Mutacja	SWAP	SWAP
elitesNum	1	1
REPAIRp	70%	0%
OPTp	30%	0%
REDISTp	80%	0%

W każdej fazie testowane są warianty różniące się *tylko* strojonym parametrem; zwycięski wariant staje się nową bazą dla kolejnej fazy.

Niezależne potoki zapewniają *fair comparison*: każdy wariant dobiera swoje optymalne parametry, a nie tylko parametry dopasowane do drugiego.

5.2 Porównanie końcowe

Porównanie przeprowadzono na 12 instancjach Solomona, $N = 5$ niezależnych uruchomień każdej konfiguracji, z kryterium zatrzymania opartym na stagnacji: algorytm zatrzymuje się, gdy przez 10 000 kolejnych pokoleń nie nastąpiła poprawa najlepszego rozwiązania (twardy limit: 1 000 000 wywołań funkcji celu).

5.3 Środowisko obliczeniowe

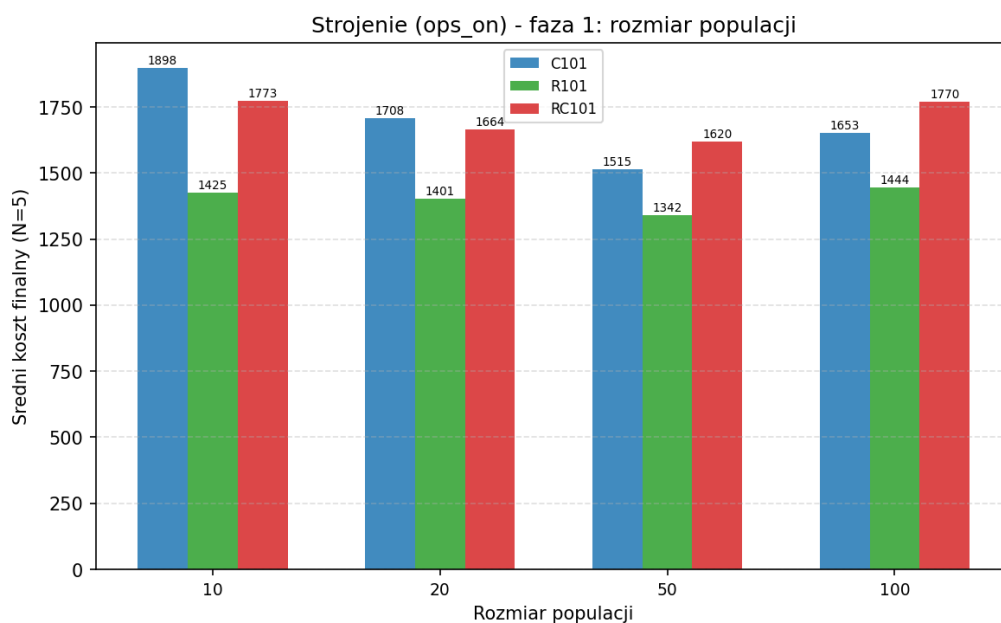
Implementacja w C++17, skompilowana z flagami `-O3 -march=native -flto -ffast-math`. Generator losowy: `std::mt19937` (Mersenne Twister).

6 Strojenie parametrów – potok ops_on

6.1 Faza 1: rozmiar populacji

Tabela 2: Faza 1 (ops_on) – strojenie popSize ($B = 300\,000$, $N = 5$)

popSize	C101	R101	RC101	OVERALL
10	1642	1425	1772	1613
20	1707	1401	1663	1591
50	1515	1341	1620	1492
100	1652	1443	1769	1622



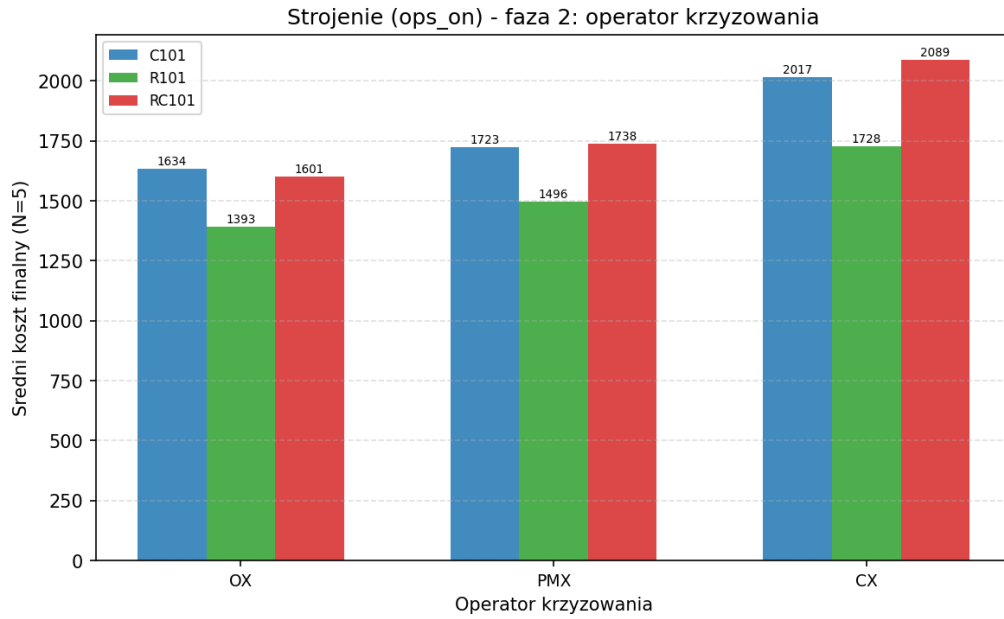
Rysunek 1: Faza 1 (ops_on): wpływ rozmiaru populacji na średni koszt finalny.

Wnioski: Populacja 50 wygrała we wszystkich instancjach. Mała populacja (10, 20) oznacza mało pokoleń przy stałym budżecie – operatory niestandardowe nie mają czasu na dopracowanie tras. Zbyt duża (100) spowalnia konwergencję. Przyjęto popSize=50.

6.2 Faza 2: operator krzyżowania

Tabela 3: Faza 2 (ops_on) – typ krzyżowania ($N = 5$)

Operator	C101	R101	RC101	OVERALL
OX	1634	1392	1600	1542
PMX	1723	1496	1737	1652
CX	2017	1728	2088	1944



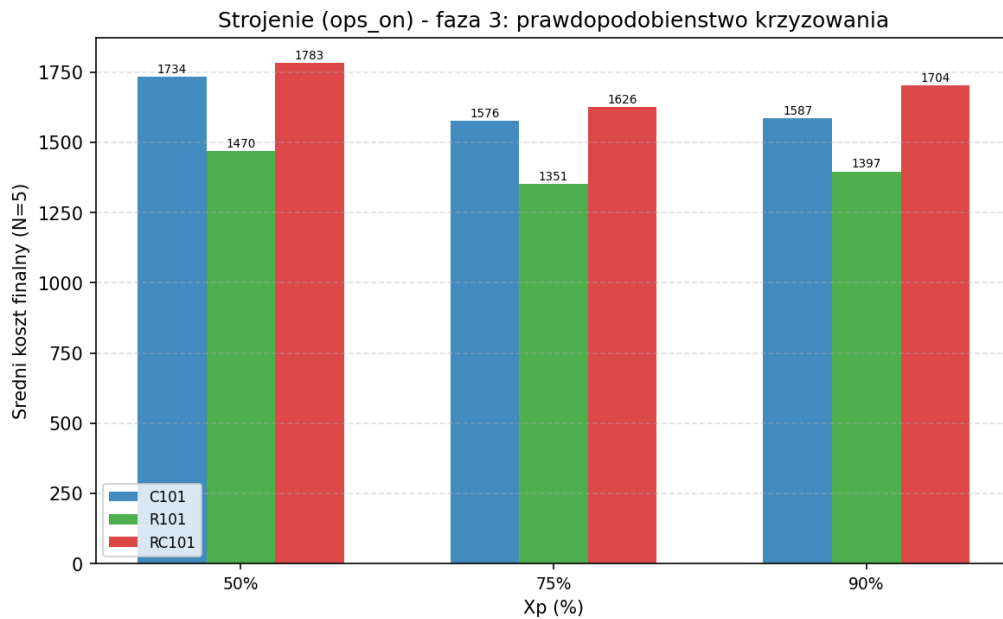
Rysunek 2: Faza 2 (ops_on): porównanie operatorów krzyżowania.

Wnioski: OX wygrał wyraźnie. CX wypadł najgorzej – operator cykliczny zachowuje pozycje absolutne, co przy problemach permutacyjnych z ważną kolejnością względną jest niekorzystne. OX zachowuje kolejność względną klientów z jednego rodzica, co jest semantycznie spójne z kodowaniem tras. Przyjęto OX.

6.3 Faza 3: prawdopodobieństwo krzyżowania (X_p)

Tabela 4: Faza 3 (ops_on) – strojenie X_p ($N = 5$)

X_p	C101	R101	RC101	OVERALL
50%	1734	1469	1783	1662
75%	1576	1351	1626	1518
90%	1586	1396	1703	1562



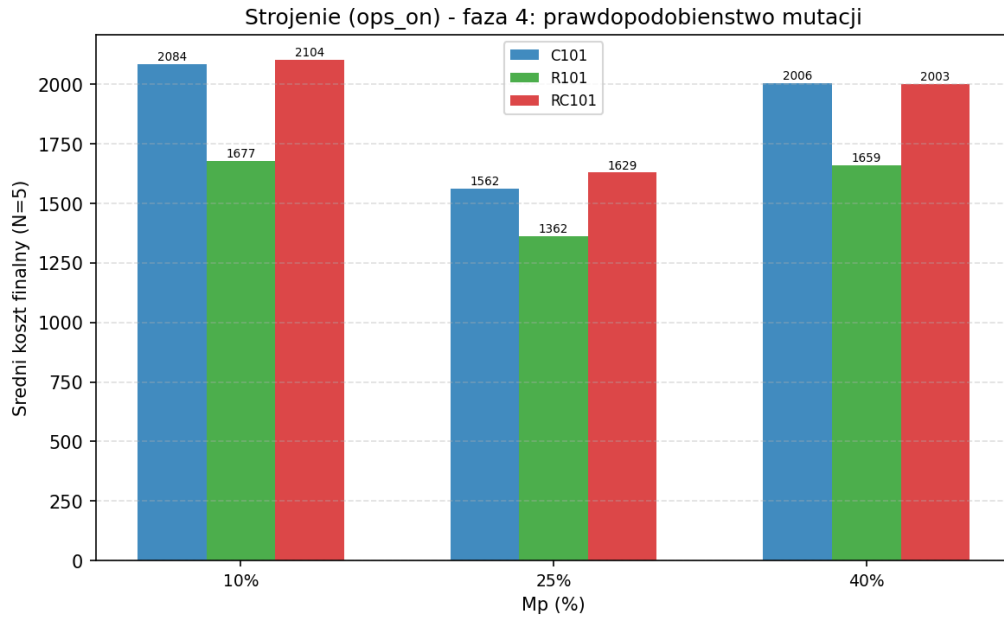
Rysunek 3: Faza 3 (ops_on): wpływ prawdopodobieństwa krzyżowania.

Wnioski: $X_p=75\%$ daje najlepsze wyniki. Zbyt rzadkie krzyżowanie (50%) ogranicza rekombinację – algorytm zachowuje się zbyt podobnie do prostej mutacji. Zbyt częste (90%) niszczy dopracowane trasy szybciej, niż operatory naprawcze zdążają je odbudować. Przyjęto $X_p=75$.

6.4 Faza 4: prawdopodobieństwo mutacji (M_p)

Tabela 5: Faza 4 (ops_on) – strojenie M_p ($N = 5$)

M_p	C101	R101	RC101	OVERALL
10%	2084	1677	2103	1954
25%	1561	1362	1629	1517
40%	2006	1659	2002	1889



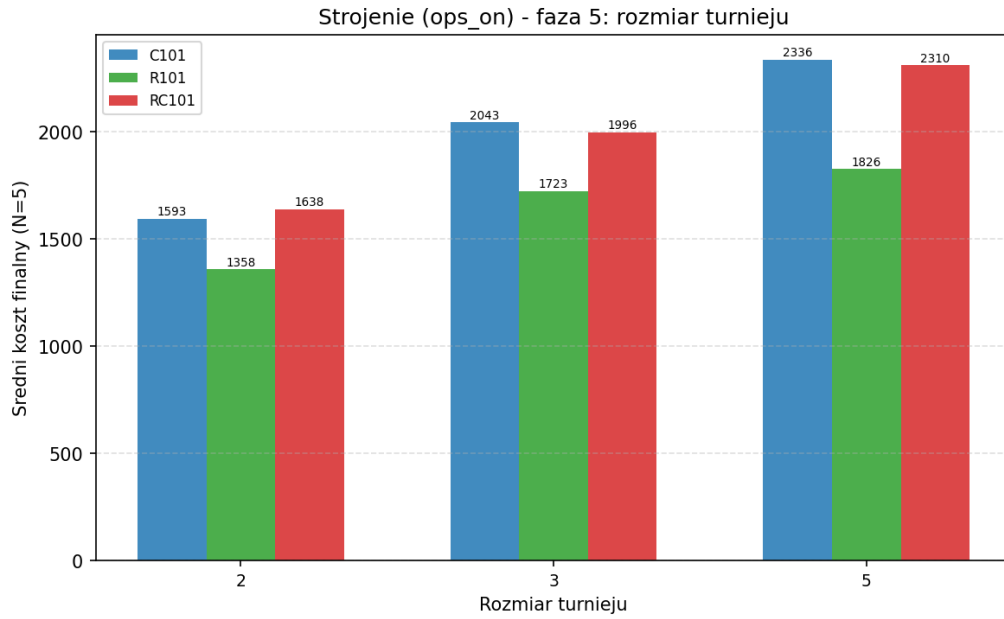
Rysunek 4: Faza 4 (ops_on): wpływ prawdopodobieństwa mutacji.

Wnioski: $Mp=25\%$ wygrało we wszystkich instancjach z dużym marginesem. Niska mutacja (10%) powoduje zbyt szybką utratę różnorodności populacji. Wysoka (40%) losowo niszczy strukturę tras – operator Repair i OptimizeTracks muszą naprawiać nie tylko błędy wynikające z krzyżowania, ale też ze zbyt agresywnej mutacji, co przekracza ich możliwości w dostępnym budżecie. Przyjęto $Mp=25$.

6.5 Faza 5: rozmiar turnieju (turSize)

Tabela 6: Faza 5 (ops_on) – strojenie turSize ($N = 5$)

turSize	C101	R101	RC101	OVERALL
2	1593	1357	1637	1529
3	2043	1722	1995	1920
5	2335	1825	2309	2157



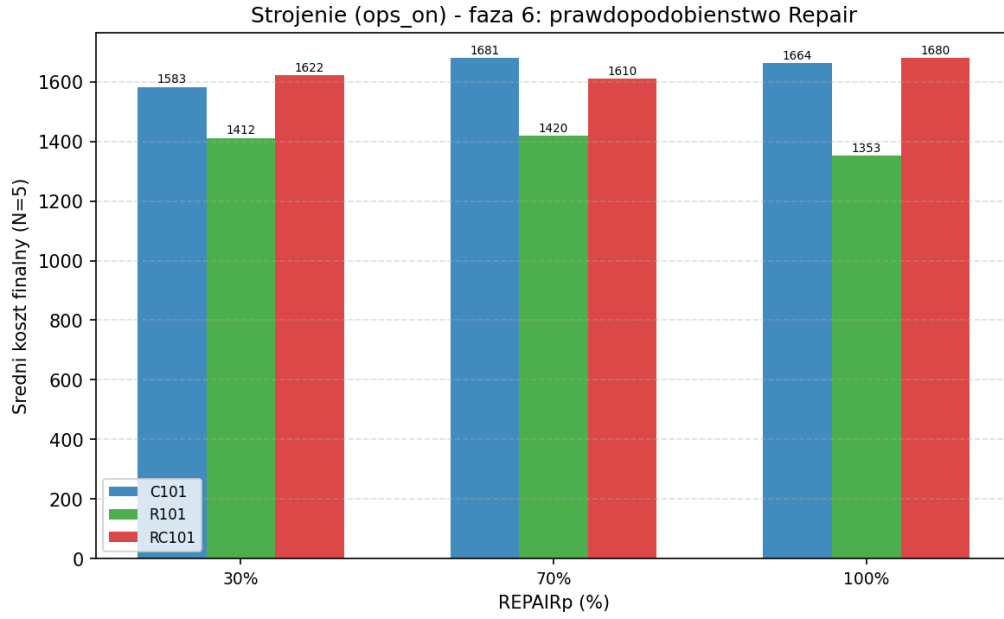
Rysunek 5: Faza 5 (ops_on): wpływ rozmiaru turnieju.

Wnioski: Mały turniej ($k = 2$) wygrał wyraźnie. Duży turniej ($k = 5$) powoduje zbyt dużą presję selekcyjną – populacja szybko ulega homogenizacji wokół lokalnego optimum, a operatory niestandardowe nie mogą wygenerować dostatecznie różnorodnych rozwiązań startowych. Przyjęto `turSize=2`.

6.6 Faza 6: prawdopodobieństwo Repair (REPAIRp)

Tabela 7: Faza 6 (ops_on) – strojenie REPAIRp ($N = 5$)

REPAIRp	C101	R101	RC101	OVERALL
30%	1583	1411	1622	1539
70%	1680	1420	1609	1570
100%	1663	1353	1680	1565



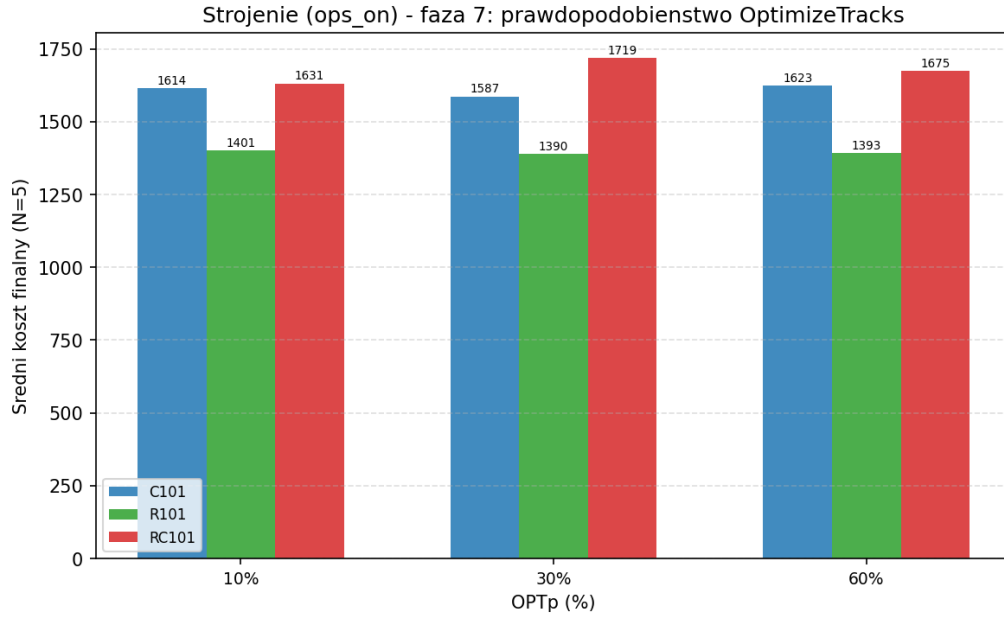
Rysunek 6: Faza 6 (ops_on): wpływ prawdopodobieństwa operatora Repair.

Wnioski: Niższe REPAIRp (30%) okazuje się lepsze – operator Repair jest kosztowny obliczeniowo (skanuje cały genom); zbyt częste jego stosowanie zmniejsza liczbę pokoleń w budżecie. Przyjęto REPAIRp=30.

6.7 Faza 7: prawdopodobieństwo OptimizeTracks (OPTp)

Tabela 8: Faza 7 (ops_on) – strojenie OPTp ($N = 5$)

OPTp	C101	R101	RC101	OVERALL
10%	1614	1401	1631	1548
30%	1586	1390	1719	1565
60%	1623	1393	1675	1563



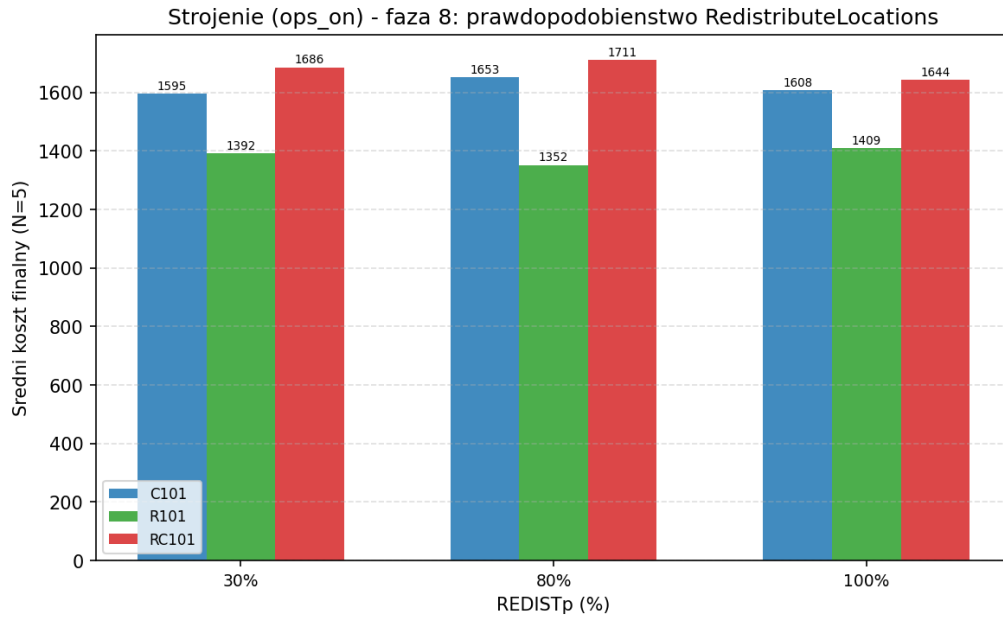
Rysunek 7: Faza 7 (ops_on): wpływ prawdopodobieństwa operatora OptimizeTracks.

Wnioski: Wyniki faz 6 i 7 są zbliżone – różnice mieszczą się w granicach zmienności statystycznej przy $N = 5$. OPT=10% wygrało globalnie; podobnie jak Repair, zbyt częste stosowanie OptimizeTracks kosztuje pokolenia. Przyjęto OPTp=10.

6.8 Faza 8: prawdopodobieństwo RedistributeLocations (REDISTp)

Tabela 9: Faza 8 (ops_on) – strojenie REDISTp ($N = 5$)

REDISTp	C101	R101	RC101	OVERALL
30%	1594	1391	1685	1557
80%	1652	1351	1711	1572
100%	1607	1409	1644	1553



Rysunek 8: Faza 8 (ops_on): wpływ prawdopodobieństwa operatora RedistributeLocations.

Wnioski: REDISTp=100% wygrało ogólnie – RedistributeLocations jest lżejszy obliczeniowo niż Repair/OPT, więc stosowanie go przy każdej mutacji nie nadmiernie obciąża budżetu. Przyjęto REDISTp=100.

6.9 Najlepsze parametry – potok ops_on

Tabela 10: Optymalne parametry EA z operatorami niestandardowymi

Parametr	Wartość
popSize	50
Krzyżowanie	OX
Xp	75%
Mp	25%
turSize	2
elitesNum	1
REPAIRp	30%
OPTp	10%
REDISTp	100%

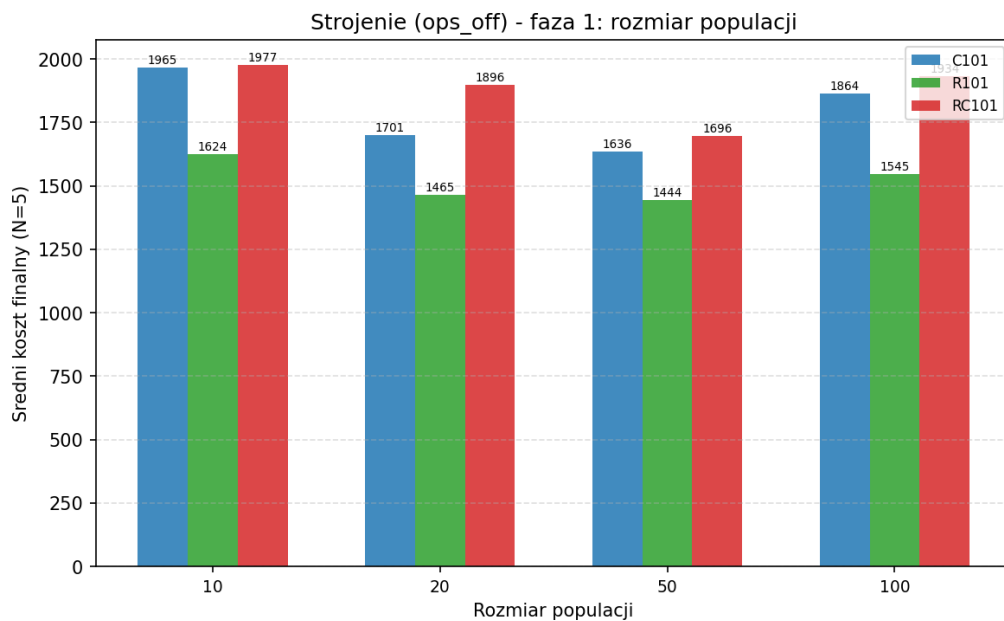
7 Strojenie parametrów – potok ops_off

Potok ops_off stroi parametry EA bazowego (bez operatorów niestandardowych) w 5 fazach. Każda faza startuje od wartości wybranej w poprzedniej fazie.

7.1 Faza 1: rozmiar populacji

Tabela 11: Faza 1 (ops_off) – strojenie popSize ($B = 300\,000$, $N = 5$)

popSize	C101	R101	RC101	OVERALL
10	1964	1624	1976	1855
20	1700	1464	1896	1687
50	1635	1443	1695	1591
100	1864	1545	1933	1780

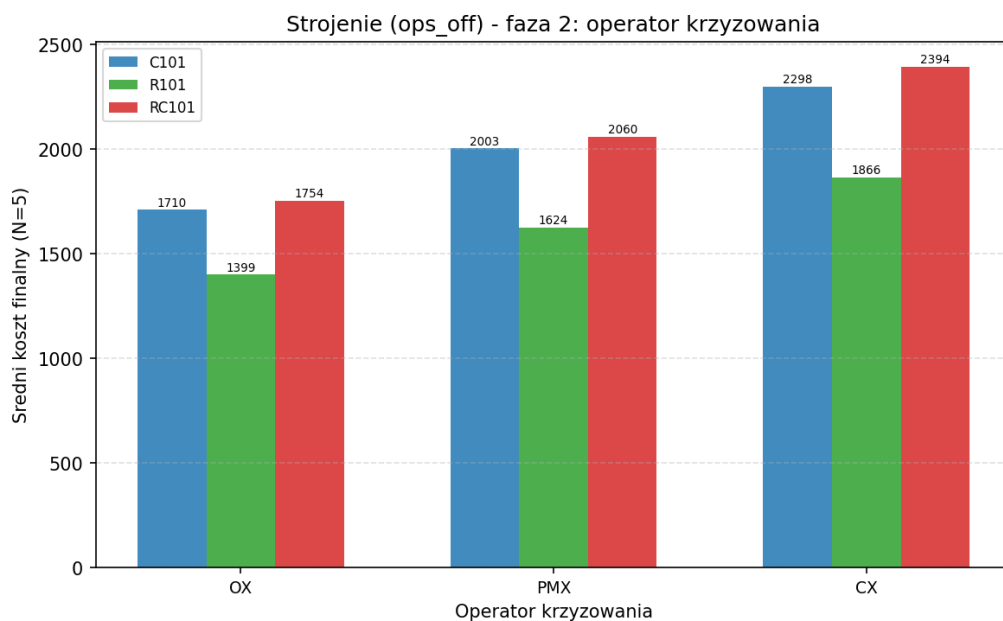


Rysunek 9: Faza 1 (ops_off): wpływ rozmiaru populacji.

7.2 Faza 2: operator krzyżowania

Tabela 12: Faza 2 (ops_off) – typ krzyżowania ($N = 5$)

Operator	C101	R101	RC101	OVERALL
OX	1709	1399	1754	1621
PMX	2002	1624	2060	1895
CX	2297	1866	2393	2185

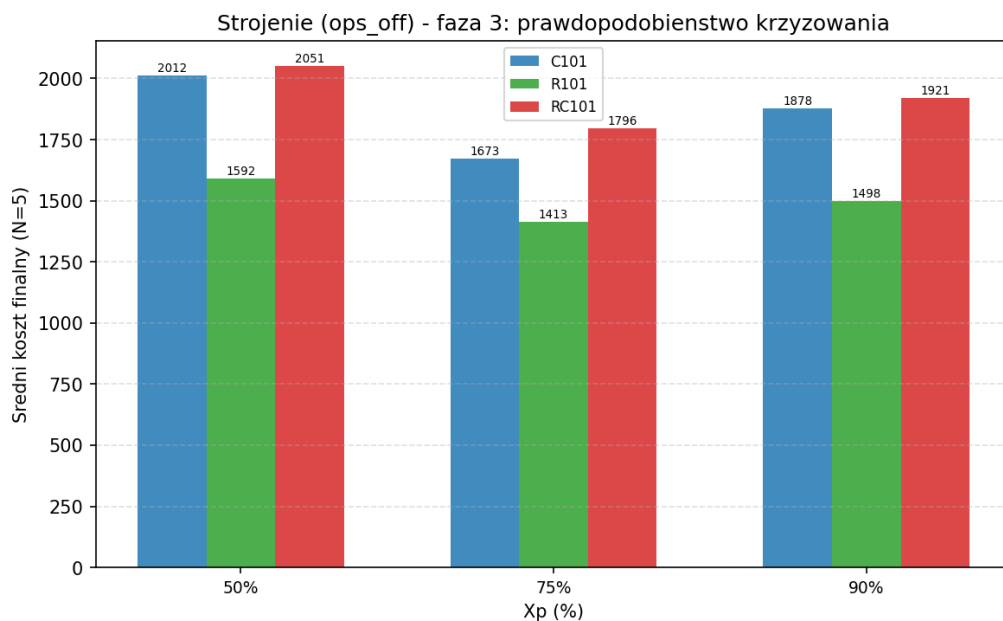


Rysunek 10: Faza 2 (ops_off): porównanie operatorów krzyżowania.

7.3 Faza 3: prawdopodobieństwo krzyżowania

Tabela 13: Faza 3 (ops_off) – strojenie X_p ($N = 5$)

X_p	C101	R101	RC101	OVERALL
50%	2012	1592	2051	1885
75%	1673	1412	1796	1627
90%	1878	1497	1920	1765

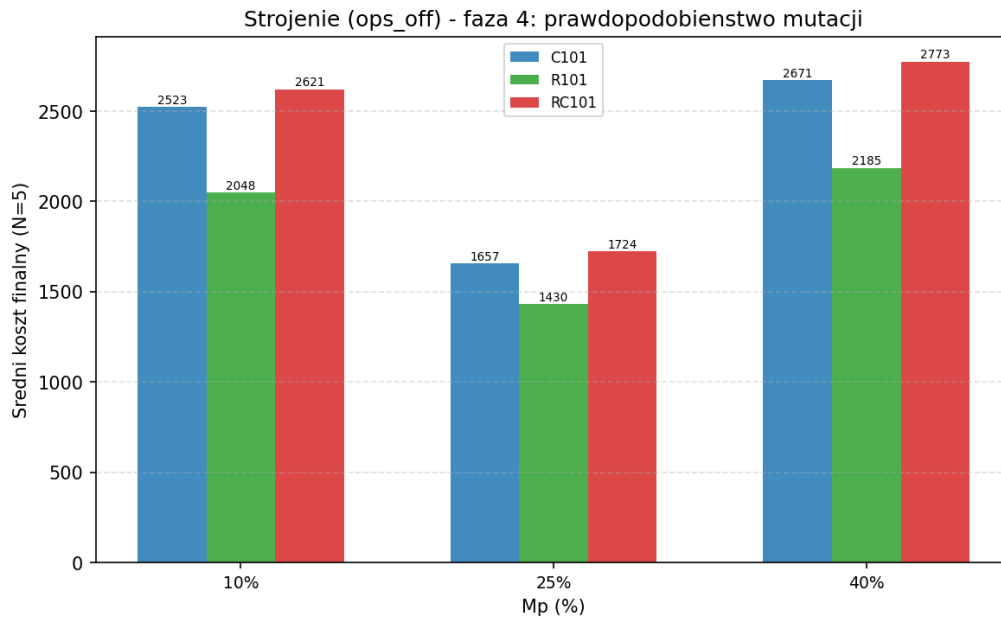


Rysunek 11: Faza 3 (ops_off): wpływ prawdopodobieństwa krzyżowania.

7.4 Faza 4: prawdopodobieństwo mutacji

Tabela 14: Faza 4 (ops_off) – strojenie M_p ($N = 5$)

M_p	C101	R101	RC101	OVERALL
10%	2522	2047	2621	2397
25%	1656	1429	1723	1603
40%	2670	2184	2773	2542



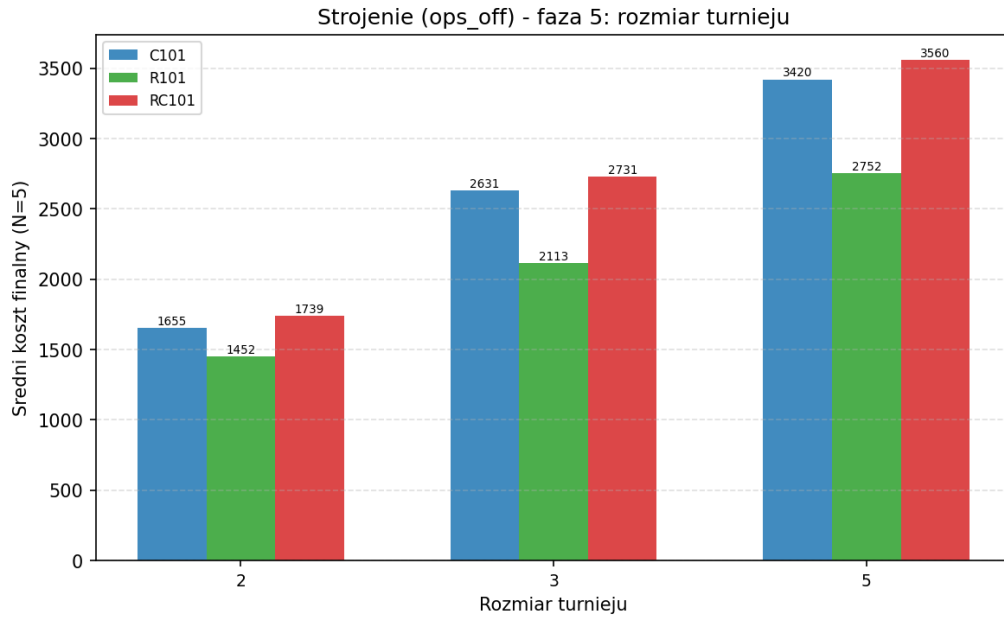
Rysunek 12: Faza 4 (ops_off): wpływ prawdopodobieństwa mutacji.

Wnioski: W wariancie bez operatorów niestandardowych wrażliwość na M_p jest znacznie większa – wynik przy $M_p=10\%$ i $M_p=40\%$ jest dramatycznie gorszy niż przy 25% . Bez mechanizmów naprawczych zbyt mała mutacja powoduje utknięcie, a zbyt duża niszczy rozwiązania nieodwracalnie.

7.5 Faza 5: rozmiar turnieju

Tabela 15: Faza 5 (ops_off) – strojenie turSize ($N = 5$)

turSize	C101	R101	RC101	OVERALL
2	1655	1451	1738	1615
3	2630	2112	2730	2491
5	3419	2752	3560	3244



Rysunek 13: Faza 5 (ops_off): wpływ rozmiaru turnieju.

Wnioski: Rozmiar turnieju ma w variancie bez operatorów efekt katastrofalny przy $k > 2$. Duża presja selekcyjna bez mechanizmów naprawczych prowadzi do szybkiej degeneracji populacji.

7.6 Najlepsze parametry – potok ops_off

Tabela 16: Optymalne parametry EA bazowego (bez operatorów niestandardowych)

Parametr	Wartość
popSize	50
Krzyżowanie	OX
Xp	75%
Mp	25%
turSize	2
elitesNum	1

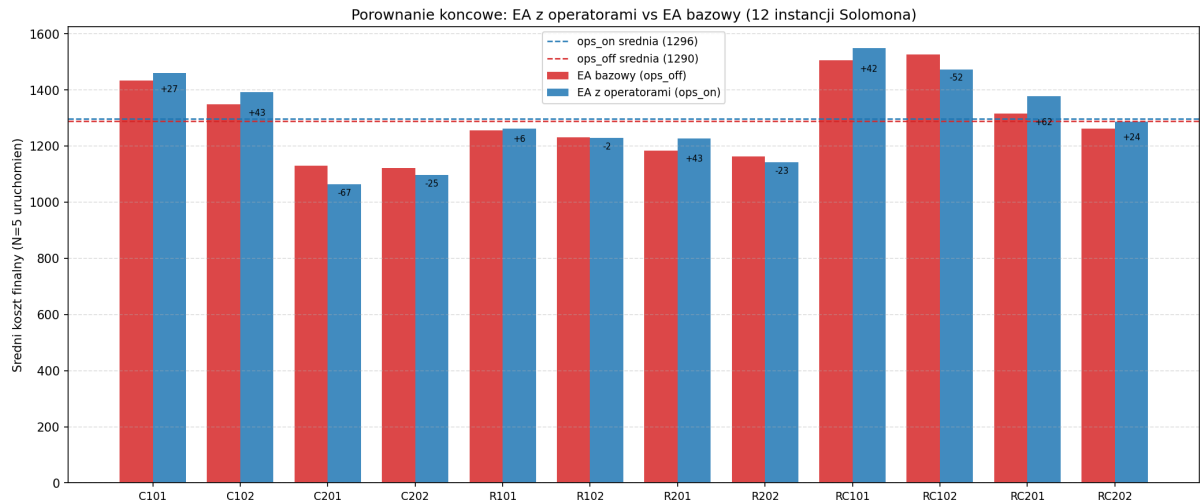
Oba potoki niezależnie wybrały identyczne parametry bazowe, co potwierdza ich robustność i wskazuje, że wybory te nie są artefaktem obecności lub braku operatorów niestandardowych.

8 Porównanie końcowe

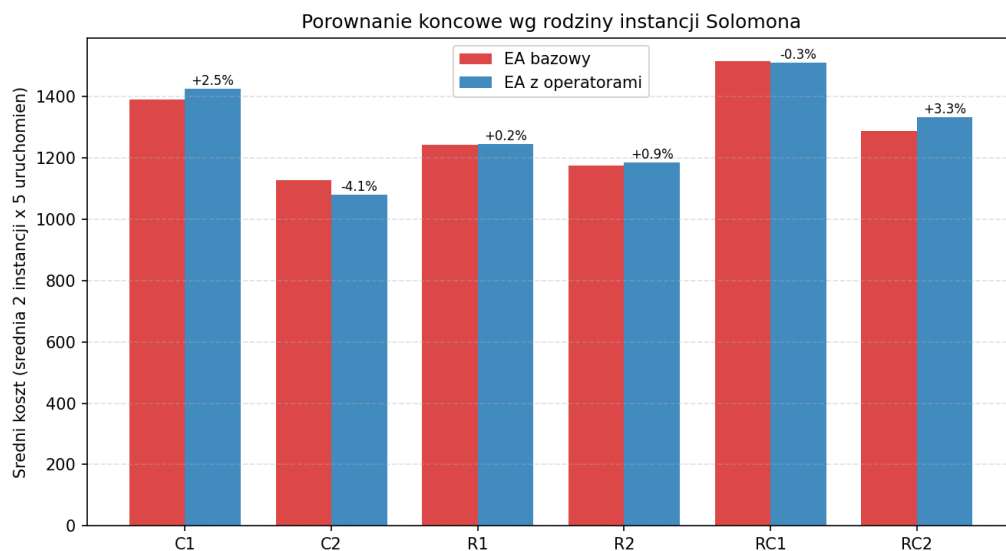
Porównanie przeprowadzono z optymalnymi konfiguracjami obu potoków na 12 instancjach Solomona. Kryterium zatrzymania: brak poprawy przez 10 000 kolejnych pokoleń lub limit 1 000 000 wywołań funkcji celu. $N = 10$ niezależnych uruchomień na instancję.

Tabela 17: Wyniki porównania końcowego – średni koszt, odchylenie standardowe i najlepszy wynik z 10 uruchomień

Instancja	ops_on			ops_off			Zwycięzca
	avg	std	best	avg	std	best	
C101	1459	76	1344	1432	101	1312	ops_off (−1,9%)
C102	1391	42	1327	1348	67	1231	ops_off (−3,1%)
C201	1063	44	988	1131	72	1027	ops_on (−6,0%)
C202	1096	80	1008	1122	75	1027	ops_on (−2,3%)
R101	1262	45	1193	1255	53	1200	ops_off (−0,6%)
R102	1229	64	1154	1231	36	1187	ops_on (−0,2%)
R201	1227	38	1150	1184	31	1137	ops_off (−3,5%)
R202	1141	47	1063	1164	51	1108	ops_on (−2,0%)
RC101	1548	91	1400	1505	67	1403	ops_off (−2,8%)
RC102	1473	51	1366	1526	107	1347	ops_on (−3,5%)
RC201	1377	45	1325	1315	73	1208	ops_off (−4,5%)
RC202	1286	38	1234	1262	65	1175	ops_off (−1,9%)
OVERALL	1296	–	–	1290	–	–	ops_off (−0,5%)

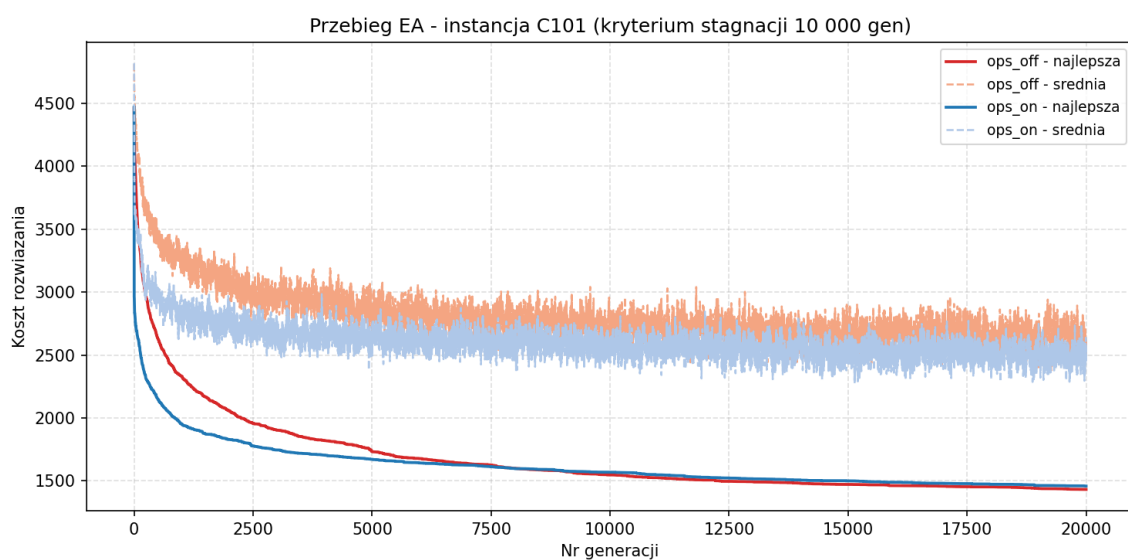


Rysunek 14: Porównanie końcowe: ops_on vs ops_off dla 12 instancji Solomona. Liczby między słupkami pokazują bezwzględną różnicę kosztów.

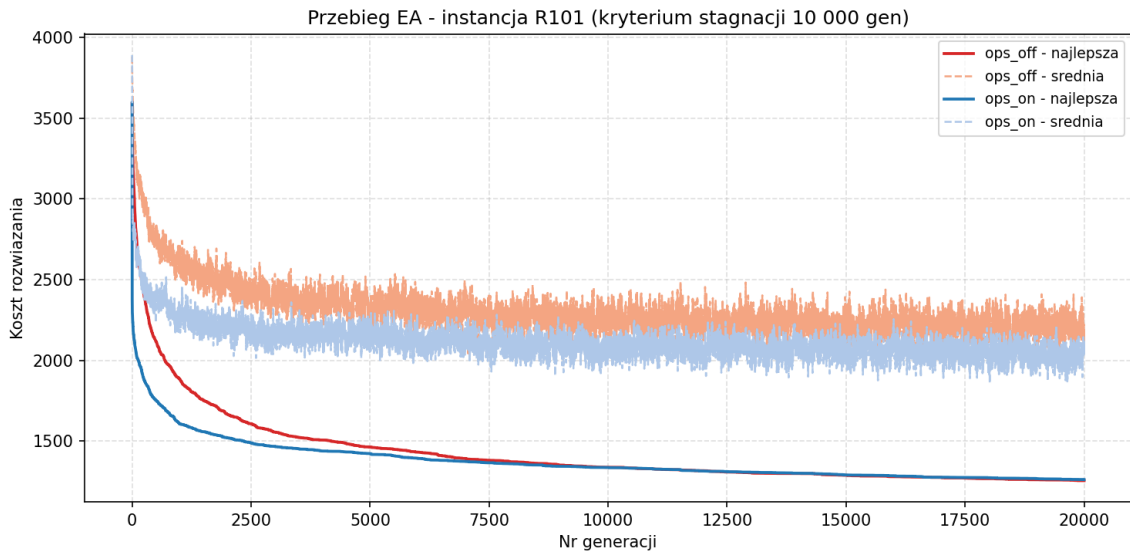


Rysunek 15: Porównanie końcowe pogrupowane wg rodziny instancji. Procenty pokazują względną różnicę ops_on względem ops_off.

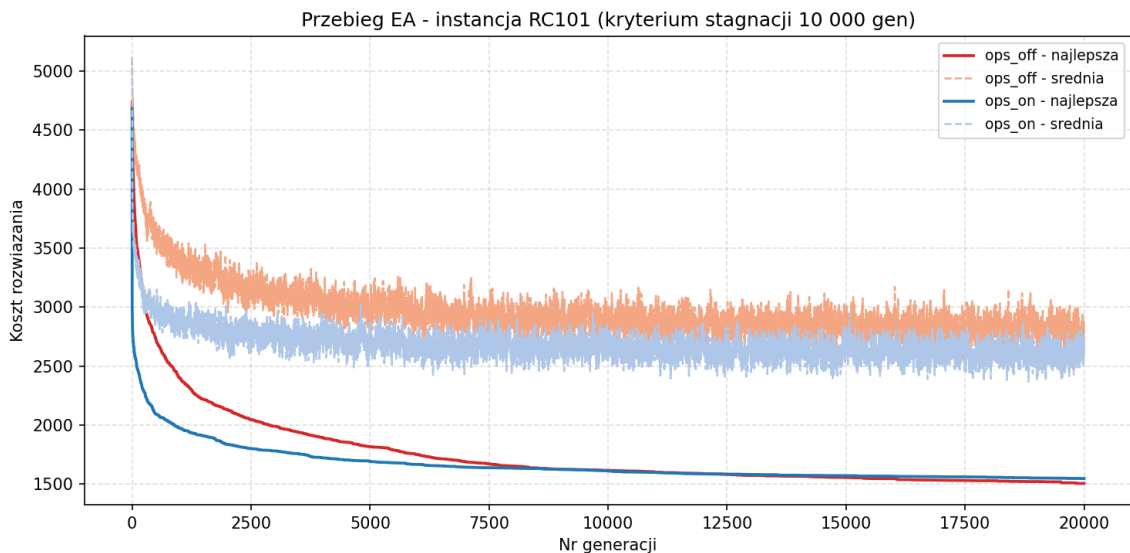
8.1 Przebiegi algorytmu



Rysunek 16: Przebieg EA dla instancji C101 – uśredniony przebieg 10 uruchomień.



Rysunek 17: Przebieg EA dla instancji R101.



Rysunek 18: Przebieg EA dla instancji RC101.

9 Analiza wyników i wnioski

9.1 Wpływ operatorów niestandardowych

EA bazowy (ops_off) osiągnął globalnie nieco lepszy wynik niż EA z operatorami (ops_on): średni koszt OVERALL 1290 vs 1296 (różnica 0,5%). ops_off wygrał na 7 z 12 instancji. Wyniki są jednak niejednorodne i zależą silnie od rodziny instancji:

- **C1 (clustered, wąskie TW):** ops_off wygrywa obie instancje (1432 vs 1459 dla C101, 1348 vs 1391 dla C102). Wąskie okna czasowe ograniczają skuteczność operatora Repair – naruszenia są trudne do naprawienia bez cofania całego przydziału klientów do tras.
- **C2 (clustered, szerokie TW):** ops_on wygrywa obie instancje wyraźnie (o 6% dla C201: 1063 vs 1131). Szerokie okna dają operatorom swobodę działania – OptimizeTracks i Redi-

tributeLocations mogą efektywnie przekształcać trasy w klastrach bez ryzyka naruszenia TW.

- **R1/R2 (random):** wyniki mieszane – każda konfiguracja wygrywa po 2 instancje. Różnice są małe (0,2–3,5%), co sugeruje, że operatory nie dają znaczącej przewagi na losowo rozmieszczonych klientach.
- **RC1/RC2 (mixed):** ops_off wygrywa 3 z 4 instancji (RC101, RC201, RC202), ops_on wygrywa tylko RC102. Największa przewaga ops_off widoczna dla RC201 (1315 vs 1377, 4,5%).

9.2 Kluczowe obserwacje

1. **Operatory niestandardowe nie poprawiają wyników globalnie.** ops_off wygrywa na 7 z 12 instancji i osiąga nieco lepszy OVERALL (1290 vs 1296, różnica 0,5%). Wynik jest zaskakujący i wskazuje, że operatory nie zawsze pomagają – ich skuteczność zależy od struktury instancji. ops_on wygrywa wyraźnie tylko na rodzinie C2 (szerokie okna).
2. **Parametry EA są niemal identyczne w obu potokach.** popSize=50, OX, Xp=75%, Mp=25%, turSize=2 okazały się optymalne niezależnie od obecności operatorów niestandardowych. Wskazuje to na robustność tych wyborów dla CVRPTW.
3. **Wrażliwość na turSize jest dramatycznie większa bez operatorów.** Przy ops_off turSize=3 daje wynik o 54% gorszy od turSize=2, podczas gdy przy ops_on ta różnica wynosi 25%. Operatory niestandardowe pełnią rolę bufora zabezpieczającego przed degeneracją populacji przy dużej presji selekcyjnej.
4. **Operatory niestandardowe preferują niskie prawdopodobieństwa.** Optymalne wartości to REPAIRp=30%, OPTp=10%, REDISTp=100%. Dwa pierwsze operatory są kosztowne obliczeniowo – zbyt częste stosowanie konsumuje budżet bez proporcjonalnej poprawy jakości. RedistributeLocations jest lżejszy i można go stosować zawsze.
5. **Przebiegi zbieżności pokazują różnicę w charakterze eksploracji.** ops_on zbiega szybciej w pierwszych generacjach (operatory natychmiast naprawiają błędy), ale ops_off może dogonić przy długim horyzoncie – szczególnie widoczne dla C101 i R101.

9.3 Porównanie z najlepszymi znanymi rozwiązaniami (BKS)

BKS (*Best Known Solution* – najlepsze znane rozwiązanie) to najlepszy wynik uzyskany kiedykolwiek dla danej instancji benchmarkowej przez dowolny algorytm opisany w literaturze naukowej. Wartości BKS dla benchmarku Solomona publikowane są i aktualizowane przez SINTEF – norweski instytut badań przemysłowych prowadzący oficjalny ranking wyników dla VRPTW (sintef.no/top/vrptw).

Wartości BKS wymagają **pełnej dopuszczalności** – wszystkie okna czasowe muszą być spełnione bez naruszeń. W niniejszej implementacji zastosowano miękkie ograniczenia z małą karą (LATE_ARRIVAL_PENALTY = 0,01), co pozwala algorytmowi na naruszanie okien czasowych kosztem minimalnej kary.

Tabela 18: Porównanie z BKS – wyniki obu konfiguracji

Instancja	BKS	ops_on avg	gap (%)	ops_off avg	gap (%)
C101	828,94	1459	+76,0%	1432	+72,7%
C102	828,94	1391	+67,8%	1348	+62,6%
C201	591,56	1063	+79,7%	1131	+91,2%
C202	591,56	1096	+85,3%	1122	+89,7%
R101	1650,80	1262	−23,5%	1255	−23,9%
R102	1486,12	1229	−17,3%	1231	−17,2%
R201	1252,37	1227	−2,0%	1184	−5,5%
R202	1191,70	1141	−4,3%	1164	−2,3%
RC101	1696,95	1548	−8,8%	1505	−11,3%
RC102	1554,75	1473	−5,2%	1526	−1,8%
RC201	1406,94	1377	−2,1%	1315	−6,5%
RC202	1365,65	1286	−5,8%	1262	−7,6%

Interpretacja wyników względem BKS:

- **Instancje C (klastrowe):** nasze wyniki są o 63–91% gorsze od BKS. Instancje C mają wąskie okna czasowe – algorytm ze słabą karą za spóźnienie nie jest wystarczająco motywowany do ich respektowania i generuje trasy o mniejszym dystansie kosztem wielu naruszeń.
- **Instancje R i RC:** nasze wyniki są *poniżej* BKS (do −24%). Jest to bezpośredni dowód naruszania okien czasowych – minimalna odległość dopuszczalnych tras dla R101 wynosi 1650,80, więc wynik 1262 jest fizycznie niemożliwy bez naruszeń. Mała kara (0,01) sprawia, że algorytm ignoruje okna czasowe i minimalizuje tylko dystans.

Podsumowanie: zastosowana penalizacja jest zbyt mała, aby wymusić dopuszczalność. Porównanie bezpośrednie z BKS jest dlatego niemożliwe. Wartością eksperymentu jest porównanie *względne* ops_on vs ops_off w identycznych warunkach.

9.4 Ograniczenia badania

- Zastosowana penalizacja (współczynnik 0,01 za spóźnienie) jest zbyt mała, by zapewnić dopuszczalność rozwiązań. Wyniki R/RC poniżej BKS jednoznacznie wskazują na naruszenia okien czasowych.
- Liczba powtórzeń $N = 10$ daje ograniczoną moc statystyczną – różnice poniżej 2% mogą nie być istotne statystycznie.
- Strojenie metodą one-factor-at-a-time nie gwarantuje globalnego optimum w przestrzeni parametrów – interakcje między parametrami nie są badane.
- Z wykresów zbieżności widać, że algorytm zbiega bardzo szybko – już po kilkuset pokoleniach krzywa najlepszego rozwiązania praktycznie się wypłaszcza. Wskazuje to na przedwczesną utratę różnorodności populacji i zbyt wczesną eksploatację lokalnego optimum. Potencjalne kierunki poprawy to zwiększenie rozmiaru populacji (co spowalnia utratę różnorodności), wprowadzenie mechanizmów utrzymania różnorodności (np. fitness sharing, crowding) lub zastosowanie restartów populacji po wykryciu stagnacji.